

Compiler Optimizations for Higher-Order Automatic Differentiation in JAX

Anonymous Authors¹

Abstract

This document provides a basic paper template and submission guidelines. Abstracts must be a single paragraph, ideally between 4–6 sentences long. Gross violations will trigger corrections at the camera-ready phase.

1. Introduction

Automatic differentiation (AD) has become a central tool in machine learning, particularly in deep learning, enabling the training of complex models in frameworks like PyTorch and TensorFlow. AD computes derivatives by systematically applying the chain rule to the primitive operations of a program, producing derivatives that are exact up to floating-point arithmetic. While most widely used AD systems are optimized for first-order derivatives, many emerging applications require second- and higher-order derivatives, including Hessians, Jacobian-vector products, and Laplacians.

Higher-order AD is expensive, but its cost depends not only on the derivative being computed, but also on how the derivative program is generated and optimized. Different higher-order AD strategies, such as nested first-order AD (e.g. repeatedly applying first-order forward/reverse mode AD) or Taylor-mode AD (e.g. computing higher-order derivatives via a Taylor series expansion), can produce mathematically equivalent derivatives while lowering to structurally different tensor programs. These differences affect the opportunities available to downstream compiler optimizations.

In this paper, we study higher-order AD as a compiler optimization problem. We lower several higher-order AD methods to StableHLO, apply several optimization passes, and execute both original and optimized programs through XLA. We make the following contributions:

1. We present a pipeline for comparing higher-order AD methods before and after compiler optimization.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

2. We evaluate repeated first-order AD compositions and Taylor-mode AD on higher-order derivative workloads.
3. We show that compiler optimization can substantially improve performance, reducing runtime by up to 3.6x and compile overhead by up to 1.8x on Laplacian benchmarks.
4. We demonstrate that compiler optimizations can change which AD strategy performs best, suggesting that AD method selection should account for downstream compiler optimizability.

2. Higher-Order AD Methods Produce Different Compiler IR

In general, AD constructs a new computation that evaluates derivatives of the original computation. For example, suppose we write a simple JAX function:

```
def f(x):  
    return x * x + 1.0  
  
g = jax.grad(f)
```

Mathematically, $g(x) = 2x$. However, from the compiler's point of view, `jax.grad(f)` does not simply record this symbolic formula; instead, it constructs a new program whose operations compute the derivative of `f`. This distinction becomes especially important for higher-order AD: two methods may compute the same mathematical derivative while generating substantially different compiler IR.

In JAX, higher-order derivatives can be computed using transformations such as `jax.hessian`, `jax.jvp`, `jax.grad`, or `jax.experimental.jet`. These transformed programs can be traced into JAX's internal *jaxpr* representation, a functional IR that exposes the primitive operations and data dependencies of the staged computation. JAX then lowers *jaxpr* to StableHLO, a portable tensor-program IR in the OpenXLA ecosystem. StableHLO is then passed to XLA, which produces an executable for CPU, GPU, or TPU backends. StableHLO represents the generated computation using high-level operations such as `dot_general`, `add`, `multiply`, `tanh`, `reshape`, `broadcast`, and `reduce`.

Crucially, the choice of higher-order AD method determines the StableHLO structure exposed to the compiler, and therefore which optimizations become available. Below, we discuss several higher-order AD methods.

2.1. Nested First-Order AD

A common strategy for computing higher-order derivatives is to repeatedly compose first-order AD transformations. This approach is appealing because it reuses mature first-order AD infrastructure: second-order quantities can be computed by differentiating a program that already computes a derivative.

The drawback is that the generated derivative program may be much larger than the final quantity of interest. Since each AD transformation is applied to the program produced by the previous one, nested AD can duplicate intermediate computations and fail to expose higher-order structure such as symmetry or sparsity. For example, when computing a Laplacian, only the diagonal entries of the Hessian are needed, but a nested AD formulation may still generate IR resembling a larger Hessian computation. This leads to redundant tensor operations and increased compilation overhead.

2.1.1. HESSIAN

Take computing the Laplacian (the trace of the Hessian) of a function, for example. One approach is to first materialize the full $n \times n$ Hessian via `jax.hessian`, which is implemented as `jacfwd(jacrev(f))`, and then take its trace:

```
hessian = jax.hessian(scalar_fn)(x)
return jnp.trace(hessian)
```

Mathematically, this constructs the full matrix (left) and returns the following (right):

$$H(x) = \nabla^2 f(x) \in \mathbb{R}^{d \times d} \quad \text{tr}(H(x)) = \sum_i H_{ii}(x).$$

At the IR level, this method materializes a Hessian-shaped tensor before applying an explicit trace. This exposes a recognizable trace pattern to the compiler: a compiler pass could use this structure to canonicalize the trace operation and remove off-diagonal computations that do not contribute to the result.

2.1.2. HESSIAN-VECTOR PRODUCT

Another way to compute the Laplacian is to use Hessian-vector products rather than explicitly invoking a Hessian transformation. Hessian-vector products are a specialized instance of nested first-order AD. In particular,

`jax.jvp(jax.grad(f), (x,), (v,))` computes a Hessian-vector product using forward-over-reverse AD: reverse mode constructs the gradient computation, and forward mode differentiates that gradient computation in the direction v .

For a direction vector v , consider the following:

```
grad_fn = jax.grad(scalar_fn)
_, hvp = jax.jvp(grad_fn, (x,), (v,))
return jnp.dot(v, hvp)
```

Since the Jacobian of ∇f is the Hessian, the tangent output of this JVP is

$$\text{JVP}(\nabla f)(x; v) = \nabla^2 f(x)v.$$

Taking the inner product with the same direction gives the second directional derivative

$$v^\top \nabla^2 f(x)v.$$

When v is a standard basis vector e_i , this quantity is exactly the i -th diagonal entry of the Hessian:

$$e_i^\top \nabla^2 f(x)e_i = H_{ii}(x).$$

Thus, evaluating this computation over all basis directions and summing the results yields the Laplacian,

$$\Delta f(x) = \sum_i e_i^\top \nabla^2 f(x)e_i.$$

2.2. Taylor-Mode AD

An alternative strategy computes higher-order derivatives via truncated Taylor series, implemented in JAX as `jax.experimental.jet`. For each basis direction e_i , Taylor-mode considers the scalar curve

$$g_i(t) = f(x + te_i).$$

and extracts

$$g_i''(0) = e_i^\top \nabla^2 f(x)e_i = H_{ii}(x).$$

Summing these second derivatives over all basis directions gives the Laplacian:

$$\Delta f(x) = \sum_i H_{ii}(x).$$

At the IR level, Taylor-mode exposes explicit Taylor-coefficient propagation. This introduces coefficient-normalization operations, such as constant `chlo.lgamma` terms, as well as many structurally zero Taylor components, creating opportunities for constant folding, zero propagation, and dead-code elimination.

3. Optimization Passes

We ask ourselves the following research question:

How do we effectively optimize code which requires the use of higher-order derivatives?

We believe that the answer to this question lies in designing or adapting compiler techniques can exploit the high-level structure exposed by derivative operators. Specifically, we implement the following passes described below. Figure 1 shows the full pipeline of where our optimizations lie.

Direct Input-Laplacian Recurrence for Tanh MLPs.

Our first optimization targets the scalar tanh-MLP Laplacian benchmark. Rather than lowering a generic higher-order AD expression such as `hessian(f)`, repeated `jvp` applications, or Taylor-mode AD, we generate code for a direct forward recurrence that computes the input Laplacian of the network.

For each layer, the ordinary forward pass computes

$$z_\ell = a_{\ell-1}W_\ell + b_\ell, \quad a_\ell = \tanh(z_\ell).$$

The optimized recurrence augments this forward pass with two input-derivative quantities: the Jacobian

$$G_\ell = \frac{\partial a_\ell}{\partial x}$$

and the componentwise input Laplacian

$$\lambda_\ell = \Delta a_\ell.$$

Thus each layer propagates the triple

$$(a_\ell, G_\ell, \lambda_\ell),$$

where a_ℓ is the activation, G_ℓ records first derivatives with respect to the original input, and λ_ℓ records the Laplacian of each activation component.

The recurrence is initialized by

$$a_0 = x, \quad G_0 = I, \quad \lambda_0 = 0.$$

For the affine part of layer ℓ , derivatives propagate linearly:

$$G_\ell^z = G_{\ell-1}W_\ell, \quad \lambda_\ell^z = \lambda_{\ell-1}W_\ell.$$

The tanh nonlinearity is then handled by the chain rule:

$$a_\ell = \tanh(z_\ell),$$

$$G_\ell = \tanh'(z_\ell) \odot G_\ell^z,$$

and

$$\lambda_\ell = \tanh''(z_\ell) \odot \|G_\ell^z\|_{\text{row}}^2 + \tanh'(z_\ell) \odot \lambda_\ell^z.$$

Here $\|G_\ell^z\|_{\text{row}}^2$ denotes the squared norm of the input-gradient vector for each activation component. Using $\tanh'(z) = 1 - \tanh^2(z)$ and $\tanh''(z) = -2 \tanh(z)(1 - \tanh^2(z))$, the recurrence lowers to ordinary tensor operations such as `dot_general`, `tanh`, `multiply`, `add`, and `reduce`.

This computes the same input Laplacian as the corresponding higher-order AD program, but avoids materializing the full Hessian or constructing a large nested derivative computation.

Constant Propagation Higher-order AD, especially Taylor-mode AD, can introduce scalar coefficient computations that are independent of the program input. In the StableHLO emitted by `jax.experimental.jet`, these appear as constant `chlo.lgamma` calls, followed by `exponential`, `convert`, `broadcast_in_dim`, and arithmetic operations used to scale Taylor coefficients. Our constant propagation pass evaluates such scalar and splat constant expressions before dispatching the program to XLA.

For example, a derivative-generated StableHLO program may contain:

```
// Before constant propagation
%cst = stablehlo.constant dense<1.00e+01> : tensor<f32>
%0 = chlo.lgamma %cst : tensor<f32> -> tensor<f32>
```

The pass rewrites this to:

```
// After constant propagation
%0 = stablehlo.constant dense<1.280183e+01> : tensor<f32>
```

The same pass also folds chains of constant operations. For example, Jet often emits coefficient-normalization code of the form:

```
%cst = stablehlo.constant dense<3.00e+00> : tensor<f32>
%0 = chlo.lgamma %cst : tensor<f32> -> tensor<f32>
%1 = stablehlo.exponential %0 : tensor<f32>
%2 = stablehlo.broadcast_in_dim %1, dims = []
    : (tensor<f32>) -> tensor<256x3xf32>
```

After constant propagation, the `lgamma` and `exponential` operations are replaced by constants, leaving a simpler StableHLO program with fewer Taylor-mode bookkeeping operations.

Structural Zero Elimination Higher-order AD also introduces tangent, cotangent, or Taylor-coefficient values that are zero by construction. These zeros can be broadcast, reshaped, and propagated through arithmetic before later passes remove them. Our structural zero elimination pass tracks syntactically known zero SSA values in StableHLO and rewrites operations whose results are also known to be zero.

For example, Jet-generated code often contains broadcasted zero tensors that are immediately added into a derivative computation:

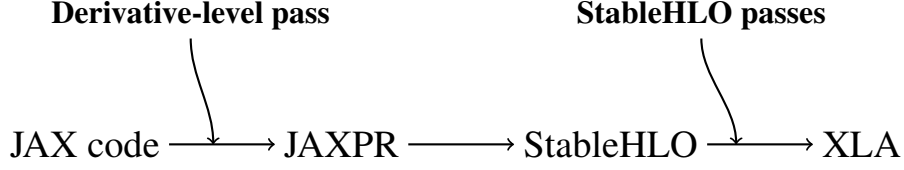


Figure 1. Compilation pipeline showing where our compiler passes are inserted. The derivative-level pass rewrites the higher-order derivative computation before lowering, while the StableHLO passes optimize the lowered IR before dispatching to XLA.

```

// Before structural zero elimination
%z = stablehlo.constant dense<0.00e+00> : tensor<f32>
%0 = stablehlo.broadcast_in_dim %z, dims = []
    : (tensor<f32>) -> tensor<256x3x256xf32>
%1 = stablehlo.add %0, %x : tensor<256x3x256xf32>
    
```

Since %0 is known to be zero, the addition can be replaced by an alias to %x:

```

// After structural zero elimination
%1 = %x
    
```

4. Evaluation

In this section, we evaluate our approach on two benchmarks:

4.1. MLP Laplacian

4.2. Physics-Informed NN for Poisson’s Equation

4.3.

5. Discussion

5.1. Compiler Optimization Can Change the Best AD Method

5.2. Limitations and Future Work

5.3. Related Work

6. Conclusion

6.1. Hessian Diagonal by JVP-of-Grad: MLP LAPLACIAN_JVP_GRAD

The second method computes Hessian-vector products using a JVP of the gradient:

```

grad_fn = jax.grad(scalar_fn)
_, hvp = jax.jvp(grad_fn, (x,), (direction,))
return jnp.dot(direction, hvp)
    
```

Since the Jacobian of ∇f is the Hessian,

$$\text{JVP}(\nabla f)(x; v) = \nabla^2 f(x) v.$$

Using a basis direction e_i gives

$$e_i^\top \nabla^2 f(x) e_i = H_{ii}(x).$$

Therefore, running this computation for each basis vector and summing the results computes the Laplacian.

Unlike the dense Hessian method, this approach does not materialize the full Hessian. Instead, it produces repeated Hessian-vector-product computations, one per basis direction:

$$e_0^\top H e_0 + e_1^\top H e_1 + e_2^\top H e_2.$$

This exposes repeated derivative subgraphs, basis-vector constants, zero tangents, dot products, broadcasts, and reshapes. These repeated structures can create opportunities for common subexpression elimination and canonicalization.

6.2. Repeated First-Order AD Composition: MLP LAPLACIAN_AUTO

The `MLP_LAPLACIAN_AUTO` method also computes higher-order derivatives by composing first-order AD transformations. Examples include forward-over-reverse, reverse-over-forward, and reverse-over-reverse AD. These methods are mathematically equivalent for smooth functions, but they may lower to different StableHLO programs.

For an MLP layer

$$h_{\ell+1} = \sigma(W_\ell h_\ell + b_\ell),$$

forward-mode structure introduces tangent computations such as

$$\dot{z}_\ell = W_\ell \dot{h}_\ell, \quad \dot{h}_{\ell+1} = \sigma'(z_\ell) \dot{z}_\ell.$$

Reverse-mode structure introduces cotangent computations such as

$$\bar{h}_\ell = W_\ell^\top \bar{z}_\ell.$$

Thus, repeated first-order AD can expose explicit linear-algebraic derivative structure: tangent propagation, cotangent propagation, transposed weight matrices, activation derivatives, and repeated primal computations. Although this can make the initial IR larger, it may also make optimization opportunities more visible to standard compiler passes.

6.3. Taylor-Mode Higher-Order AD:

MLP_LAPLACIAN_JET

The `MLP_LAPLACIAN_JET` method uses Taylor-mode AD. For each basis direction e_i , it considers the one-dimensional curve

$$g_i(t) = f(x + te_i).$$

Then

$$g_i''(0) = e_i^\top \nabla^2 f(x) e_i = H_{ii}(x).$$

Summing these second derivatives over all basis directions gives the Laplacian.

Instead of repeatedly composing first-order AD operators, Taylor-mode propagates truncated Taylor coefficients through the program. For a second-order computation, an intermediate value is represented by coefficients such as

$$(a_0, a_1, a_2),$$

where a_0 is the primal value, a_1 is the first directional derivative, and a_2 is the second directional derivative.

For example, if

$$y = \sigma(z),$$

Taylor-mode propagates equations of the form

$$y_0 = \sigma(z_0), \quad y_1 = \sigma'(z_0)z_1, \quad y_2 = \sigma''(z_0)z_1^2 + \sigma'(z_0)z_2.$$

This exposes coefficient-wise propagation through the primal program rather than explicit Hessian tensors or reverse-mode cotangent computations.

A larger repeated-composition program may contain more redundant structure for common subexpression elimination, while a compact Taylor-mode program may have less obvious redundancy. This motivates comparing higher-order AD methods not only by their source-level formulation, but also by the StableHLO IR they produce and how effectively that IR can be optimized.

7. Electronic Submission

Submission to ICML 2025 will be entirely electronic, via a web site (not email). Information about the submission process and $\text{\LaTeX}$ templates are available on the conference web site at:

<http://icml.cc/>

The guidelines below will be enforced for initial submissions and camera-ready copies. Here is a brief summary:

- Submissions must be in PDF.
- If your paper has appendices, submit the appendix together with the main body and the references **as a**

single file. Reviewers will not look for appendices as a separate PDF file. So if you submit such an extra file, reviewers will very likely miss it.

- Page limit: The main body of the paper has to be fitted to 8 pages, excluding references and appendices; the space for the latter two is not limited in pages, but the total file size may not exceed 50MB during submission. For the final version of the paper, authors can add one extra page to the main body and the file size is limited to 20MB.
- **Do not include author information or acknowledgements** in your initial submission.
- Your paper should be in **10 point Times font**.
- Make sure your PDF file only uses Type-1 fonts.
- Place figure captions *under* the figure (and omit titles from inside the graphic file itself). Place table captions *over* the table.
- References must include page numbers whenever possible and be as complete as possible. Place multiple citations in chronological order.
- Do not alter the style template; in particular, do not compress the paper format by reducing the vertical spaces.
- Keep your abstract brief and self-contained, one paragraph and roughly 4–6 sentences. Gross violations will require correction at the camera-ready phase. The title should have content words capitalized.

7.1. Submitting Papers

Anonymous Submission: ICML uses double-blind review: no identifying author information may appear on the title page or in the paper itself. Section 8.3 gives further details.

Authors must provide their manuscripts in **PDF** format. Furthermore, please make sure that files contain only embedded Type-1 fonts (e.g., using the program `pdfonts` in linux or using File/DocumentProperties/Fonts in Acrobat). Other fonts (like Type-3) might come from graphics files imported into the document.

Authors using **Word** must convert their document to PDF. Most of the latest versions of Word have the facility to do this automatically. Submissions will not be accepted in Word format or any format other than PDF. Really. We're not joking. Don't send Word.

Those who use **$\text{\LaTeX}$** should avoid including Type-3 fonts. Those using `latex` and `dvips` may need the following two commands:

`dvips -Ppdf -tletter -G0 -o paper.ps paper.dvi` edit the header of the document themselves.
`ps2pdf paper.ps`

It is a zero following the “-G”, which tells dvips to use the `config.pdf` file. Newer $\text{\TeX}$ distributions don’t always need this option.

Using `pdflatex` rather than `latex`, often gives better results. This program avoids the Type-3 font problem, and supports more advanced features in the `microtype` package.

Graphics files should be a reasonable size, and included from an appropriate format. Use vector formats (`.eps/.pdf`) for plots, lossless bitmap formats (`.png`) for raster graphics with sharp lines, and `jpeg` for photo-like images.

The style file uses the `hyperref` package to make clickable links in documents. If this causes problems for you, add `nohyperref` as one of the options to the `icml2025` `usepackage` statement.

7.2. Submitting Final Camera-Ready Copy

The final versions of papers accepted for publication should follow the same format and naming convention as initial submissions, except that author information (names and affiliations) should be given. See Section 8.3.2 for formatting instructions.

The footnote, “Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.” must be modified to “*Proceedings of the 42nd International Conference on Machine Learning*, Vancouver, Canada, PMLR 267, 2025. Copyright 2025 by the author(s).”

For those using the $\text{\LaTeX}$ style file, this change (and others) is handled automatically by simply changing `\usepackage{icml2025}` to

```
\usepackage[accepted]{icml2025}
```

Authors using **Word** must edit the footnote on the first page of the document themselves.

Camera-ready copies should have the title of the paper as running head on each page except the first one. The running title consists of a single line centered above a horizontal rule which is 1 point thick. The running head should be centered, bold and in 9 point type. The rule should be 10 points above the main text. For those using the $\text{\LaTeX}$ style file, the original title is automatically set as running head using the `fancyhdr` package which is included in the ICML 2025 style file package. In case that the original title exceeds the size restrictions, a shorter form can be supplied by using

```
\icmltitlerunning{...}
```

just before `\begin{document}`. Authors using **Word** must

8. Format of the Paper

All submissions must follow the specified format.

8.1. Dimensions

The text of the paper should be formatted in two columns, with an overall width of 6.75 inches, height of 9.0 inches, and 0.25 inches between the columns. The left margin should be 0.75 inches and the top margin 1.0 inch (2.54 cm). The right and bottom margins will depend on whether you print on US letter or A4 paper, but all final versions must be produced for US letter size. Do not write anything on the margins.

The paper body should be set in 10 point type with a vertical spacing of 11 points. Please use Times typeface throughout the text.

8.2. Title

The paper title should be set in 14 point bold type and centered between two horizontal rules that are 1 point thick, with 1.0 inch between the top rule and the top edge of the page. Capitalize the first letter of content words and put the rest of the title in lower case.

8.3. Author Information for Submission

ICML uses double-blind review, so author information must not appear. If you are using $\text{\LaTeX}$ and the `icml2025.sty` file, use `\icmlauthor{...}` to specify authors and `\icmlaffiliation{...}` to specify affiliations. (Read the TeX code used to produce this document for an example usage.) The author information will not be printed unless `accepted` is passed as an argument to the style file. Submissions that include the author information will not be reviewed.

8.3.1. SELF-CITATIONS

If you are citing published papers for which you are an author, refer to yourself in the third person. In particular, do not use phrases that reveal your identity (e.g., “in previous work (Langley, 2000), we have shown ...”).

Do not anonymize citations in the reference section. The only exception are manuscripts that are not yet published (e.g., under submission). If you choose to refer to such unpublished manuscripts (Author, 2021), anonymized copies have to be submitted as Supplementary Material via OpenReview. However, keep in mind that an ICML paper should be self contained and should contain sufficient detail for the reviewers to evaluate the work. In particular, reviewers are

not required to look at the Supplementary Material when writing their review (they are not required to look at more than the first 8 pages of the submitted document).

8.3.2. CAMERA-READY AUTHOR INFORMATION

If a paper is accepted, a final camera-ready copy must be prepared. For camera-ready papers, author information should start 0.3 inches below the bottom rule surrounding the title. The authors' names should appear in 10 point bold type, in a row, separated by white space, and centered. Author names should not be broken across lines. Unbolded superscripted numbers, starting 1, should be used to refer to affiliations.

Affiliations should be numbered in the order of appearance. A single footnote block of text should be used to list all the affiliations. (Academic affiliations should list Department, University, City, State/Region, Country. Similarly for industrial affiliations.)

Each distinct affiliations should be listed once. If an author has multiple affiliations, multiple superscripts should be placed after the name, separated by thin spaces. If the authors would like to highlight equal contribution by multiple first authors, those authors should have an asterisk placed after their name in superscript, and the term “*Equal contribution” should be placed in the footnote block ahead of the list of affiliations. A list of corresponding authors and their emails (in the format Full Name <email@domain.com>) can follow the list of affiliations. Ideally only one or two names should be listed.

A sample file with author names is included in the ICML2025 style file package. Turn on the `[accepted]` option to the stylefile to see the names rendered. All of the guidelines above are implemented by the $\text{\LaTeX}$ style file.

8.4. Abstract

The paper abstract should begin in the left column, 0.4 inches below the final address. The heading ‘Abstract’ should be centered, bold, and in 11 point type. The abstract body should use 10 point type, with a vertical spacing of 11 points, and should be indented 0.25 inches more than normal on left-hand and right-hand margins. Insert 0.4 inches of blank space after the body. Keep your abstract brief and self-contained, limiting it to one paragraph and roughly 4–6 sentences. Gross violations will require correction at the camera-ready phase.

8.5. Partitioning the Text

You should organize your paper into sections and paragraphs to help readers place a structure on the material and understand its contributions.

8.5.1. SECTIONS AND SUBSECTIONS

Section headings should be numbered, flush left, and set in 11 pt bold type with the content words capitalized. Leave 0.25 inches of space before the heading and 0.15 inches after the heading.

Similarly, subsection headings should be numbered, flush left, and set in 10 pt bold type with the content words capitalized. Leave 0.2 inches of space before the heading and 0.13 inches afterward.

Finally, subsubsection headings should be numbered, flush left, and set in 10 pt small caps with the content words capitalized. Leave 0.18 inches of space before the heading and 0.1 inches after the heading.

Please use no more than three levels of headings.

8.5.2. PARAGRAPHS AND FOOTNOTES

Within each section or subsection, you should further partition the paper into paragraphs. Do not indent the first line of a given paragraph, but insert a blank line between succeeding ones.

You can use footnotes¹ to provide readers with additional information about a topic without interrupting the flow of the paper. Indicate footnotes with a number in the text where the point is most relevant. Place the footnote in 9 point type at the bottom of the column in which it appears. Precede the first footnote in a column with a horizontal rule of 0.8 inches.²

8.6. Figures

You may want to include figures in the paper to illustrate your approach and results. Such artwork should be centered, legible, and separated from the text. Lines should be dark and at least 0.5 points thick for purposes of reproduction, and text should not appear on a gray background.

Label all distinct components of each figure. If the figure takes the form of a graph, then give a name for each axis and include a legend that briefly describes each curve. Do not include a title inside the figure; instead, the caption should serve this function.

Number figures sequentially, placing the figure number and caption *after* the graphics, with at least 0.1 inches of space before the caption and 0.1 inches after it, as in Figure 2. The figure caption should be set in 9 point type and centered unless it runs two or more lines, in which case it should be flush left. You may float figures to the top or bottom of a

¹Footnotes should be complete sentences.

²Multiple footnotes can appear in each column, in the same order as they appear in the text, but spread them across columns and pages if possible.

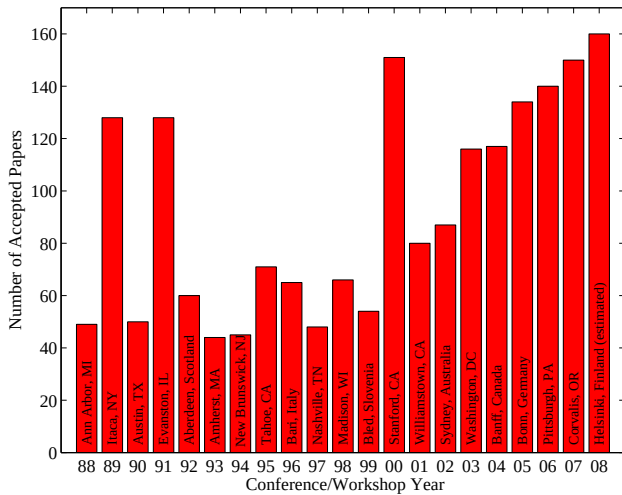


Figure 2. Historical locations and number of accepted papers for International Machine Learning Conferences (ICML 1993 – ICML 2008) and International Workshops on Machine Learning (ML 1988 – ML 1992). At the time this figure was produced, the number of accepted papers for ICML 2008 was unknown and instead estimated.

Algorithm 1 Bubble Sort

```

Input: data  $x_i$ , size  $m$ 
repeat
  Initialize  $noChange = true$ .
  for  $i = 1$  to  $m - 1$  do
    if  $x_i > x_{i+1}$  then
      Swap  $x_i$  and  $x_{i+1}$ 
       $noChange = false$ 
    end if
  end for
until  $noChange$  is  $true$ 

```

column, and you may set wide figures across both columns (use the environment `figure*` in `LATEX`). Always place two-column figures at the top or bottom of the page.

8.7. Algorithms

If you are using `LATEX`, please use the “algorithm” and “algorithmic” environments to format pseudocode. These require the corresponding stylefiles, `algorithm.sty` and `algorithmic.sty`, which are supplied with this package. Algorithm 1 shows an example.

8.8. Tables

You may also want to include tables that summarize material. Like figures, these should be centered, legible, and numbered consecutively. However, place the title *above* the

Table 1. Classification accuracies for naive Bayes and flexible Bayes on various data sets.

DATA SET	NAIVE	FLEXIBLE	BETTER?
BREAST	95.9 ± 0.2	96.7 ± 0.2	✓
CLEVELAND	83.3 ± 0.6	80.0 ± 0.6	×
GLASS2	61.9 ± 1.4	83.8 ± 0.7	✓
CREDIT	74.8 ± 0.5	78.3 ± 0.6	
HORSE	73.3 ± 0.9	69.7 ± 1.0	×
META	67.1 ± 0.6	76.5 ± 0.5	✓
PIMA	75.1 ± 0.6	73.9 ± 0.5	
VEHICLE	44.9 ± 0.6	61.5 ± 0.4	✓

table with at least 0.1 inches of space before the title and the same after it, as in Table 1. The table title should be set in 9 point type and centered unless it runs two or more lines, in which case it should be flush left.

Tables contain textual material, whereas figures contain graphical material. Specify the contents of each row and column in the table’s topmost row. Again, you may float tables to a column’s top or bottom, and set wide tables across both columns. Place two-column tables at the top or bottom of the page.

8.9. Theorems and such

The preferred way is to number definitions, propositions, lemmas, etc. consecutively, within sections, as shown below.

Definition 8.1. A function $f : X \rightarrow Y$ is injective if for any $x, y \in X$ different, $f(x) \neq f(y)$.

Using Theorem 8.1 we immediately get the following result:

Proposition 8.2. If f is injective mapping a set X to another set Y , the cardinality of Y is at least as large as that of X .

Proof. Left as an exercise to the reader. □

Theorem 8.3 stated next will prove to be useful.

Lemma 8.3. For any $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ injective functions, $f \circ g$ is injective.

Theorem 8.4. If $f : X \rightarrow Y$ is bijective, the cardinality of X and Y are the same.

An easy corollary of Theorem 8.4 is the following:

Corollary 8.5. If $f : X \rightarrow Y$ is bijective, the cardinality of X is at least as large as that of Y .

Assumption 8.6. The set X is finite.

Remark 8.7. According to some, it is only the finite case (cf. Theorem 8.6) that is interesting.

8.10. Citations and References

Please use APA reference format regardless of your formatter or word processor. If you rely on the L^AT_EX bibliographic facility, use `natbib.sty` and `icml2025.bst` included in the style-file package to obtain this format.

Citations within the text should include the authors' last names and year. If the authors' names are included in the sentence, place only the year in parentheses, for example when referencing Arthur Samuel's pioneering work (1959). Otherwise place the entire reference in parentheses with the authors and year separated by a comma (Samuel, 1959). List multiple references separated by semicolons (Kearns, 1989; Samuel, 1959; Mitchell, 1980). Use the 'et al.' construct only for citations with three or more authors or after listing all authors to a publication in an earlier reference (Michalski et al., 1983).

Authors should cite their own work in the third person in the initial version of their paper submitted for blind review. Please refer to Section 8.3 for detailed instructions on how to cite your own papers.

Use an unnumbered first-level section heading for the references, and use a hanging indent style, with the first line of the reference flush against the left margin and subsequent lines indented by 10 points. The references at the end of this document give examples for journal articles (Samuel, 1959), conference publications (Langley, 2000), book chapters (Newell & Rosenbloom, 1981), books (Duda et al., 2000), edited volumes (Michalski et al., 1983), technical reports (Mitchell, 1980), and dissertations (Kearns, 1989).

Alphabetize references by the surnames of the first authors, with single author entries preceding multiple author entries. Order references for the same authors by year of publication, with the earliest first. Make sure that each reference includes all relevant information (e.g., page numbers).

Please put some effort into making references complete, presentable, and consistent, e.g. use the actual current name of authors. If using bibtex, please protect capital letters of names and abbreviations in titles, for example, use `{B}ayesian` or `{L}ipschitz` in your `.bib` file.

Accessibility

Authors are kindly asked to make their submissions as accessible as possible for everyone including people with disabilities and sensory or neurological differences. Tips of how to achieve this and what to pay attention to will be provided on the conference website <http://icml.cc/>.

Software and Data

If a paper is accepted, we strongly encourage the publication of software and data with the camera-ready version of the paper whenever appropriate. This can be done by including a URL in the camera-ready copy. However, **do not** include URLs that reveal your institution or identity in your submission for review. Instead, provide an anonymous URL or upload the material as "Supplementary Material" into the OpenReview reviewing system. Note that reviewers are not required to look at this material when writing their review.

Acknowledgements

Do not include acknowledgements in the initial version of the paper submitted for blind review.

If a paper is accepted, the final camera-ready version can (and usually should) include acknowledgements. Such acknowledgements should be placed at the end of the section, in an unnumbered section that does not count towards the paper page limit. Typically, this will include thanks to reviewers who gave useful comments, to colleagues who contributed to the ideas, and to funding agencies and corporate sponsors that provided financial support.

Impact Statement

Authors are **required** to include a statement of the potential broader impact of their work, including its ethical aspects and future societal consequences. This statement should be in an unnumbered section at the end of the paper (co-located with Acknowledgements – the two may appear in either order, but both must be before References), and does not count toward the paper page limit. In many cases, where the ethical impacts and expected societal implications are those that are well established when advancing the field of Machine Learning, substantial discussion is not required, and a simple statement such as the following will suffice:

"This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here."

The above statement can be used verbatim in such cases, but we encourage authors to think about whether there is content which does warrant further discussion, as this statement will be apparent if the paper is later flagged for ethics review.

References

Author, N. N. Suppressed for anonymity, 2021.

Duda, R. O., Hart, P. E., and Stork, D. G. *Pattern Classification*. John Wiley and Sons, 2nd edition, 2000.

- Kearns, M. J. *Computational Complexity of Machine Learning*. PhD thesis, Department of Computer Science, Harvard University, 1989.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Michalski, R. S., Carbonell, J. G., and Mitchell, T. M. (eds.). *Machine Learning: An Artificial Intelligence Approach, Vol. I*. Tioga, Palo Alto, CA, 1983.
- Mitchell, T. M. The need for biases in learning generalizations. Technical report, Computer Science Department, Rutgers University, New Brunswick, MA, 1980.
- Newell, A. and Rosenbloom, P. S. Mechanisms of skill acquisition and the law of practice. In Anderson, J. R. (ed.), *Cognitive Skills and Their Acquisition*, chapter 1, pp. 1–51. Lawrence Erlbaum Associates, Inc., Hillsdale, NJ, 1981.
- Samuel, A. L. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):211–229, 1959.

A. You *can* have an appendix here.

You can have as much text here as you want. The main body must be at most 8 pages long. For the final version, one more page can be added. If you want, you can use an appendix like this one.

The `\onecolumn` command above can be kept in place if you prefer a one-column appendix, or can be removed if you prefer a two-column appendix. Apart from this possible change, the style (font size, spacing, margins, page numbering, etc.) should be kept the same as the main body.